EXP 3	Implementation of XOR with backpropagation algorithm

AIM: Understand the concept of XOR with backpropagation algorithm using python programing language.

Algorithm:

1. Initialize the neural network with random weights and biases.
2. Define the XOR training dataset.
3. Set training parameters: learning rate, number of epochs, number of hidden neurons, and activation function.
4. For each epoch:
   a. For each training sample:
      - Perform forward propagation to compute the output.
      - Calculate the error between actual and predicted output.
      - Apply backpropagation to compute gradients.
      - Update weights and biases using the learning rate.
   b. Compute the average error for the epoch.
5. Repeat until the error is minimized or maximum epochs are reached.
6. Use the trained network to predict XOR outputs.
7. End.

PROGRAM:

```
import numpy as np

# Define sigmoid activation function and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
      return x * (1 - x)

# XOR input and target data
input_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
target_data = np.array([[0], [1], [1], [0]])

# Neural network architecture
input_size = 2
hidden_size = 2
output_size = 1
learning_rate = 0.1
epochs = 10000

# Initialize weights randomly with mean 0
hidden_weights = np.random.uniform(size=(input_size, hidden_size))
output_weights = np.random.uniform(size=(hidden_size, output_size))
```

```
# Training loop
for _ in range(epochs):
    # Forward propagation
    hidden_layer_activation = np.dot(input_data, hidden_weights)
    hidden_layer_output = sigmoid(hidden_layer_activation)

    output_layer_activation = np.dot(hidden_layer_output, output_weights)
    predicted_output = sigmoid(output_layer_activation)

    # Calculate error
    error = target_data - predicted_output

    # Backpropagation
    output_delta = error * sigmoid_derivative(predicted_output)

    hidden_layer_error = output_delta.dot(output_weights.T)
    hidden_layer_delta = hidden_layer_error * sigmoid_derivative(hidden_layer_output)

    # Update weights
    output_weights += hidden_layer_output.T.dot(output_delta) * learning_rate
    hidden_weights += input_data.T.dot(hidden_layer_delta) * learning_rate

# Test the trained network
test_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
for data in test_data:
    hidden_layer_activation = np.dot(data, hidden_weights)
    hidden_layer_output = sigmoid(hidden_layer_activation)

    output_layer_activation = np.dot(hidden_layer_output, output_weights)
    predicted_output = sigmoid(output_layer_activation)

    print(f"Input: {data} Predicted Output: {predicted_output[0]}")
```

Output:

```
Input: [0 0] Predicted Output: 0.287381655624125
Input: [0 1] Predicted Output: 0.6696713061093961
Input: [1 0] Predicted Output: 0.6697648563700653
Input: [1 1] Predicted Output: 0.42466198065447125
```

Result: Thus the above program classification with discrete perception executed successfully with desired output.